1. **Write Testable Code with Moq**

Scenario

You are tasked to write a unit test code for the below scenario.

The application in which you are teamed up with, deals with a mail server communication in which your application tries to send mail to its users upon every transaction. Your role is to write unit testing the module that contains send mail functionality. You wanted to perform testing the module without sending any email.

After investigating the problem scenario, you found a solution and that is creating **mock** objects of these external dependencies in the unit testing project so that you can achieve speedier test execution and loose coupling of code.

Task1

In this task, you will create a class library that will be used for unit testing.

- Create a **Class Library (Language C#)** project using Visual Studio IDE, and name it as **CustomerCommLib.**
- Rename the default **Class1** class name as **MailSender.**
- Include the following namespaces with 'using' directive.
    - **System.Net**
    - **System.Net.Mail**
- Define an interface as follow.

```
public interface IMailSender
{
    bool SendMail(string toAddress, string message);
}
```

Source code:

CustomerCommLib/MailSender.cs

```
using System.Net;
using System.Net.Mail;
namespace CustomerCommLib
{
  public interface IMailSender
  {
    bool SendMail(string toAddress, string message);
```

```
}

public class MailSender : IMailSender
{
    private readonly string _smtpHost;
    private readonly int _smtpPort;
    private readonly string _senderAddress;
    public MailSender(string smtpHost = "smtp.example.com",
                      int smtpPort = 587,
                      string senderAddress = "no-reply@example.com")
    {
        _smtpHost = smtpHost;
        _smtpPort = smtpPort;
        _senderAddress = senderAddress;
    }
    public bool SendMail(string toAddress, string message)
    {
        try
        {
            using (var mailMessage = new MailMessage(_senderAddress, toAddress))
            {
                mailMessage.Subject = "Transaction Notification";
                mailMessage.Body = message;
                using (var smtpClient = new SmtpClient(_smtpHost, _smtpPort))
                {
                    smtpClient.Credentials = CredentialCache.DefaultNetworkCredentials;
                    smtpClient.Send(mailMessage);
                }
            }
            return true;
        }
        catch
```

```
        {
            return false;
        }
    }
}
public class CustomerCommService
{
    private readonly IMailSender _mailSender;
    public CustomerCommService(IMailSender mailSender)
    {
        _mailSender = mailSender;
    }
    public bool NotifyCustomerOfTransaction(string toAddress, string txnId)
    {
        string message = $"Your transaction {txnId} was completed successfully.";
        return _mailSender.SendMail(toAddress, message);
    }
}
}
```

CustomerCommLib.Tests/MailSenderTests.cs

```
using CustomerCommLib;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Moq;
namespace CustomerCommLib.Tests
{
    [TestClass]
    public class MailSenderTests
    {
        private Mock<IMailSender> _mockMailSender;
        private CustomerCommService _service;
        [TestInitialize]
```

```
public void Setup()
{
    _mockMailSender = new Mock<IMailSender>();
    _service = new CustomerCommService(_mockMailSender.Object);
}
[TestMethod]
public void NotifyCustomer_CallsSendMail_ReturnsTrue()
{
    _mockMailSender
        .Setup(m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()))
        .Returns(true);
    bool result = _service.NotifyCustomerOfTransaction("customer@example.com", "TXN123");
    Assert.IsTrue(result);
    _mockMailSender.Verify(
        m => m.SendMail("customer@example.com",
            "Your transaction TXN123 was completed successfully."),
        Times.Once);
}
[TestMethod]
public void NotifyCustomer_SendMailFails_ReturnsFalse()
{
    _mockMailSender
        .Setup(m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()))
        .Returns(false);
    bool result = _service.NotifyCustomerOfTransaction("customer@example.com", "TXN999");
    Assert.IsFalse(result);
    _mockMailSender.Verify(
        m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()),
        Times.Once);
}
[TestMethod]
public void NotifyCustomer_NeverSendsRealEmail()
```

```
    {
        _mockMailSender
            .Setup(m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()))
            .Returns(true);
        _service.NotifyCustomerOfTransaction("a@b.com", "TXN001");
        _mockMailSender.Verify(m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()),
Times.Once);
    }
  }
}
```

OUTPUT

```
Passed  NotifyCustomer_CallsSendMail_ReturnsTrue
Passed  NotifyCustomer_SendMailFails_ReturnsFalse
Passed  NotifyCustomer_NeverSendsRealEmail

3 Passed, 0 Failed, 0 Skipped
```

Task2

In this task, you will create unit test project which make use of NUnit framework and Moq.

- Create a new class library project called CustomerComm.Tests and add the following external dependencies to it using NuGet Package Manager.
    - NUnit
    - NUnit Test Adapter
    - Moq
- Add the references of assemblies as appropriate including CustomerCommLib.
- Write unit test code and mock the MailSender (IMailSender) class.
- Use TestFixture, OneTimeSetUp and TestCase attribute classes on top of test class, init method and test method respectively.
- Configure the mock object in such away that SendMail() method will accept any two string arguments and always return true when SendMailToCustomer() gets invoked.
- Finally assert the return value to “true”.

Source code:

CustomerCommLib/MailSender.cs

```
using System.Net;
using System.Net.Mail;
namespace CustomerCommLib
{
   public interface IMailSender
   {
      bool SendMail(string toAddress, string message);
   }
   public class MailSender : IMailSender
   {
      private readonly string _smtpHost;
      private readonly int _smtpPort;
      private readonly string _senderAddress;
```

```
public MailSender(string smtpHost = "smtp.example.com",
                  int smtpPort = 587,
                  string senderAddress = "no-reply@example.com")
{
   _smtpHost = smtpHost;
   _smtpPort = smtpPort;
   _senderAddress = senderAddress;
}
public bool SendMail(string toAddress, string message)
{
   try
   {
      using (var mailMessage = new MailMessage(_senderAddress, toAddress))
      {
         mailMessage.Subject = "Transaction Notification";
         mailMessage.Body = message;

         using (var smtpClient = new SmtpClient(_smtpHost, _smtpPort))
         {
            smtpClient.Credentials = CredentialCache.DefaultNetworkCredentials;
            smtpClient.Send(mailMessage);
         }
      }
      return true;
   }
   catch
   {
      return false;
   }
```

```
    }
  }
  public class CustomerCommService
  {
    private readonly IMailSender _mailSender;

    public CustomerCommService(IMailSender mailSender)
    {
      _mailSender = mailSender;
    }
    public bool SendMailToCustomer(string toAddress, string message)
    {
      return _mailSender.SendMail(toAddress, message);
    }
  }
}
```

CustomerComm.Tests/MailSenderNUnitTests.cs

NuGet packages required in this test project:

- NUnit
- NUnit3TestAdapter
- Moq
- Project reference: CustomerCommLib

```
using CustomerCommLib;
using Moq;
using NUnit.Framework;
namespace CustomerComm.Tests  {
  [TestFixture]
```

```
public class MailSenderNUnitTest   {
    private Mock<IMailSender> _mockMailSender;
    private CustomerCommService _customerCommService;
    [OneTimeSetUp]
    public void Init(){
        _mockMailSender = new Mock<IMailSender>();
        _mockMailSender
            .Setup(m => m.SendMail(It.IsAny<string>(), It.IsAny<string>()))
            .Returns(true);
        _customerCommService = new CustomerCommService(_mockMailSender.Object);
    }
    [TestCase("customer1@example.com", "Your order #101 has been shipped.")]
    [TestCase("customer2@example.com", "Your payment of $250 was received.")]
    [TestCase("customer3@example.com", "Your subscription has been renewed.")]
    public void SendMailToCustomer_WithAnyStrings_ReturnsTrue(string toAddress,
string message)
    {
        bool result = _customerCommService.SendMailToCustomer(toAddress, message);
        Assert.AreEqual(true, result);  } } }
```

OUTPUT

```
Passed!  - Failed:     0, Passed:     3, Skipped:     0, Total:     3

Passed
SendMailToCustomer_WithAnyStrings_ReturnsTrue("customer1@example.com","Your orde
#101 has been shipped.")
Passed
SendMailToCustomer_WithAnyStrings_ReturnsTrue("customer2@example.com","Your
payment of $250 was received.")
Passed
SendMailToCustomer_WithAnyStrings_ReturnsTrue("customer3@example.com","Your
subscription has been renewed.")

3 Passed, 0 Failed, 0 Skipped
```